

INTERNATIONAL CYBERSECURITY & DIGITAL FORENSICS

ACADEMY (ICDFA)

DIGITAL FORENSICS LABORATORY REPORT

Steganographic Data Concealment, Extraction, and Password Recovery Analysis

Laboratory Assessment - Steganography and Hidden Data Detection

Steganography and Hidden Data Analysis

Prepared by: Nebeuwa Ifeanyichukwu Raphael

Role: Junior Digital Forensic Trainee

Course Code: CIP-B102

Examination Platform: Kali Linux

Examination Dates: 31 August – 1 September 2026

1. Executive Summary

This report documents my performance of ICDFa Laboratory Assessment 4, in which I examined the principles and practical application of steganography within a digital forensics context. Acting in the role of a junior digital forensic trainee, I was tasked with preparing an authorised carrier image, concealing a controlled evidence file within it using least-significant-bit (LSB) substitution via Steghide, extracting the concealed data using the correct passphrase, verifying data integrity through SHA-256 cryptographic hashing, and testing a password-recovery workflow against the resulting stego image using StegSeek and a controlled wordlist.

My objective was to demonstrate a repeatable, evidentially sound forensic process: preserving the original carrier image, documenting every command executed, and validating that no unintended alteration occurred to the recovered evidence file. I encountered and resolved one significant acquisition obstacle during Part A, in which my initial download of the authorised carrier image returned an HTML interstitial page rather than the binary image file. I diagnosed this using the `file(1)` utility and resolved it by re-acquiring the carrier image with `gdown`, a tool purpose-built to handle Google Drive's large-file confirmation redirect.

All subsequent embedding, extraction, comparative, and password-recovery tasks were completed successfully. The SHA-256 hash of my extracted secret file matched the hash of the original file in every instance, confirming that Steghide's extraction process preserved data integrity. StegSeek correctly recovered the embedding passphrase from my controlled wordlist in both the guided exercise (Part F) and my independent task (Part G), confirming the expected behaviour of dictionary-based steganalysis against a weak, guessable passphrase.

2. Investigation Overview & Scope

2.1 Purpose and Objectives

The purpose of this exercise was to build practical competency in:

- Distinguishing steganography (concealment of the existence of data) from encryption (concealment of the content of data).
- Explaining insertion and substitution-based hiding techniques, with emphasis on least significant bit (LSB) substitution in bitmap images.
- Using Steghide to embed and extract concealed data under a controlled passphrase.
- Calculating and comparing SHA-256 cryptographic hashes to verify data integrity across acquisition, embedding, and extraction.
- Performing a controlled password dictionary attack using StegSeek to evaluate passphrase strength.
- Producing a clear, repeatable forensic report documenting every step and command executed.

2.2 Evidence Media and Environment

I conducted this examination entirely within a Kali Linux virtual environment (host kali@kali), using a dedicated working directory (~/.icdfa_lab4) created specifically for this assessment. The authorised lab carrier image was distributed via a controlled Google Drive link and was the sole source image used throughout this exercise. All secondary evidence files (secret.txt, the independent hidden-note file, wordlists, and extracted outputs) were created by me within this working directory for the sole purpose of this lab.

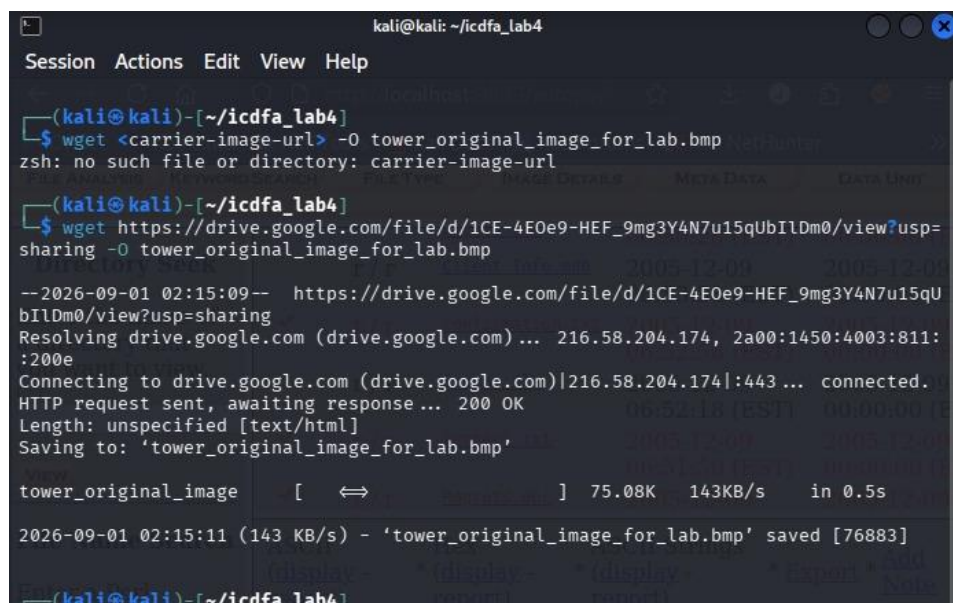
Tools used: Steghide (embedding/extraction), StegSeek 0.6 (dictionary-based passphrase recovery), sha256sum (integrity hashing), file (file-type identification), xxd (hexadecimal inspection), strings (readable-string extraction), diff (byte-for-byte comparison), and gdown (authenticated bulk-file retrieval from Google Drive).

3. Evidence Acquisition and Preservation

I began by creating a dedicated working directory, ~/.icdfa_lab4, to contain all files associated with this examination, in keeping with standard practice of isolating case evidence from unrelated data.

3.1 Initial Acquisition Attempt and Diagnosis of Failure

My first acquisition attempt used wget against the Google Drive share link directly. The download completed without an explicit error and reported a saved file of 76,883 bytes; however, the reported content length was of MIME type text/html, which was my first indication that the retrieved object was not the intended bitmap image.



```
kali@kali: ~/.icdfa_lab4
Session Actions Edit View Help

(kali@kali)~[~/.icdfa_lab4]
$ wget <carrier-image-url> -O tower_original_image_for_lab.bmp
zsh: no such file or directory: carrier-image-url

(kali@kali)~[~/.icdfa_lab4]
$ wget https://drive.google.com/file/d/1CE-4EOe9-HEF_9mg3Y4N7u15qUbIldm0/view?usp=sharing -O tower_original_image_for_lab.bmp

--2026-09-01 02:15:09-- https://drive.google.com/file/d/1CE-4EOe9-HEF_9mg3Y4N7u15qUbIldm0/view?usp=sharing
Resolving drive.google.com (drive.google.com)... 216.58.204.174, 2a00:1450:4003:811::200e
Connecting to drive.google.com (drive.google.com)|216.58.204.174|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/html]
Saving to: 'tower_original_image_for_lab.bmp'

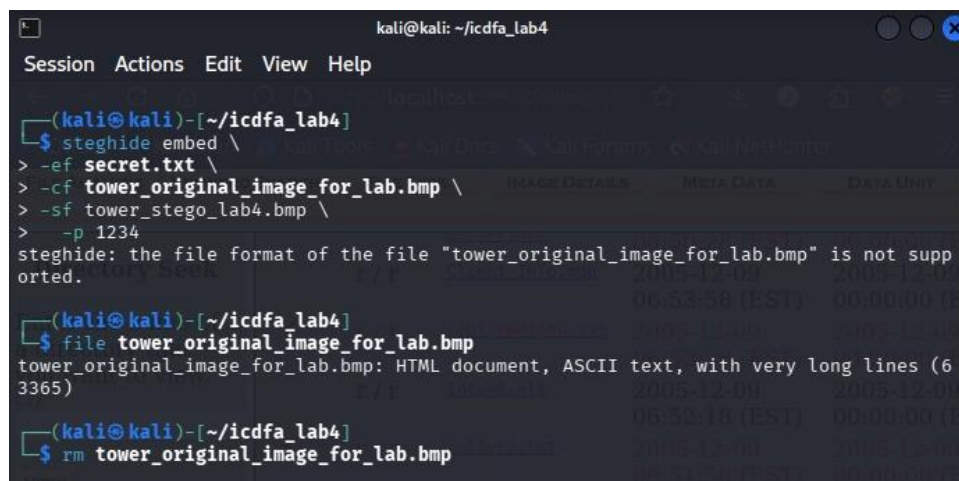
tower_original_image      [  =>  ] 75.08K  143KB/s  in 0.5s

2026-09-01 02:15:11 (143 KB/s) - 'tower_original_image_for_lab.bmp' saved [76883]

(kali@kali)~[~/.icdfa_lab4]
```

Figure 1: Initial wget acquisition attempt against the Google Drive share link. The server responded with a 200 OK and an unspecified text/html content length rather than the expected binary image.

I confirmed this diagnosis by running the `file(1)` command against the downloaded object, which identified it conclusively as an HTML document containing ASCII text with very long lines, rather than a bitmap image. This is a well-documented behaviour of Google Drive when a large file is requested via an unauthenticated, non-interactive client such as `wget`: Drive returns a virus-scan warning/confirmation interstitial page instead of streaming the file directly. Recognising that this corrupted acquisition could not be used as forensically valid carrier media, I removed the file to prevent it from being mistaken for genuine evidence.

A terminal window titled 'kali@kali: ~/icdfa_lab4' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the following commands and output:

```
(kali@kali)-[~/icdfa_lab4]
$ steghide embed \
> -ef secret.txt \
> -cf tower_original_image_for_lab.bmp \
> -sf tower_stego_lab4.bmp \
> -p 1234
steghide: the file format of the file "tower_original_image_for_lab.bmp" is not supported.

(kali@kali)-[~/icdfa_lab4]
$ file tower_original_image_for_lab.bmp
tower_original_image_for_lab.bmp: HTML document, ASCII text, with very long lines (63365)

(kali@kali)-[~/icdfa_lab4]
$ rm tower_original_image_for_lab.bmp
```

Figure 2: Confirmation via `file(1)` that the acquired object was an HTML document rather than a BMP image, evidenced further by Steghide's own refusal to process it ("the file format... is not supported"). The corrupted file was subsequently removed with `rm`.

3.2 Corrected Acquisition Using `gdown`

To obtain a forensically usable copy of the authorised carrier image, I installed `gdown`, a Python utility designed to negotiate Google Drive's large-file confirmation workflow programmatically.

```
kali@kali: ~/icdfa_lab4
Session Actions Edit View Help
(kali@kali)~-[~/icdfa_lab4]
$ pip install gdown --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting gdown
  Downloading gdown-6.1.0-py3-none-any.whl.metadata (8.3 kB)
Requirement already satisfied: beautifulsoup4 in /usr/lib/python3/dist-packages (from gdown) (4.14.3)
Requirement already satisfied: filelock in /usr/lib/python3/dist-packages (from gdown) (3.24.3)
Requirement already satisfied: requests[socks] in /usr/lib/python3/dist-packages (from gdown) (2.32.5)
Requirement already satisfied: tqdm in /usr/lib/python3/dist-packages (from gdown) (4.67.3)
Requirement already satisfied: soupsieve>=1.6.1 in /usr/lib/python3/dist-packages (from beautifulsoup4->gdown) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/lib/python3/dist-packages (from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/lib/python3/dist-packages (from requests[socks]->gdown) (3.4.4)
Requirement already satisfied: idna<4, >=2.5 in /usr/lib/python3/dist-packages (from requests[socks]->gdown) (3.11)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/lib/python3/dist-packages (from requests[socks]->gdown) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/lib/python3/dist-packages (from requests[socks]->gdown) (2026.2.25)
Requirement already satisfied: PySocks!=1.5.7, >=1.5.6 in /usr/lib/python3/dist-packages (from requests[socks]->gdown) (1.7.1)
Downloading gdown-6.1.0-py3-none-any.whl (19 kB)
Installing collected packages: gdown
Successfully installed gdown-6.1.0
```

Figure 3: Installation of gdown via pip, using the --break-system-packages flag required on Kali's externally managed Python environment.

I then re-acquired the carrier image using gdown against the file's Google Drive object identifier. The download completed successfully, retrieving 9.28 MB. I immediately validated the result with file(1), which confirmed the object as a genuine PC bitmap, Windows 3.x format image at a resolution of 1365 x 2265 pixels, 24-bit colour depth — a valid, Steghide-compatible carrier format.

```
(kali@kali)~-[~/icdfa_lab4]
$ gdown https://drive.google.com/uc?id=1CE-4E0e9-HEF_9mg3Y4N7u15qUbILdM0 -O tower_
original_image_for_lab.bmp
Downloading ...
From: https://drive.google.com/uc?id=1CE-4E0e9-HEF_9mg3Y4N7u15qUbILdM0
To: /home/kali/icdfa_lab4/tower_original_image_for_lab.bmp
100%|████████████████████████████████████████| 9.28M/9.28M [02:56<00:00, 52.6kB/s]

(kali@kali)~-[~/icdfa_lab4]
$ file tower_original_image_for_lab.bmp
tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, r
esolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54

(kali@kali)~-[~/icdfa_lab4]
$
```

Figure 4: Successful acquisition of the authorized carrier image via gdown, and confirmation via file(1) that the object is a valid 24-bit Windows 3.x bitmap (1365 x 2265 px).

3.3 Hashing and Preservation of the Original Carrier Image

Immediately upon confirming a valid acquisition, I calculated the SHA-256 hash of the original carrier image to establish an integrity baseline prior to any embedding activity. I then created a second, permission-locked copy of the original file (tower_original_PRESERVED.bmp, mode 444) so that the working copy used for embedding could not be confused with, or accidentally overwrite, the preserved original.

```
(kali@kali)-[~/icdfa_lab4]
$ sha256sum tower_original_image_for_lab.bmp | tee original_image.sha256
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp

(kali@kali)-[~/icdfa_lab4]
$ cp tower_original_image_for_lab.bmp tower_original_PRESERVED.bmp
cp: cannot create regular file 'tower_original_PRESERVED.bmp': Permission denied

(kali@kali)-[~/icdfa_lab4]
$ sudo cp tower_original_image_for_lab.bmp tower_original_PRESERVED.bmp
[sudo] password for kali:

(kali@kali)-[~/icdfa_lab4]
$ chmod 444 tower_original_PRESERVED.bmp

(kali@kali)-[~/icdfa_lab4]
$
```

Figure 5: SHA-256 hash calculation of the original carrier image, followed by creation of a permission-locked preservation copy (chmod 444) to prevent accidental modification.

Item	SHA-256 Hash / Value
Original carrier image (tower_original_image_for_lab.bmp)	6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db

4. Secret Evidence File Preparation

In accordance with Part B of the lab brief, I created a controlled evidence text file, secret.txt, containing my full name, a short controlled evidence statement, and confirmation that the file was created specifically for the ICDFA lab exercise. This file served as the payload for the guided embedding exercise in Part C. Note: my working screen captures for this stage begin at the embedding step (Section 5); the discrete hash-calculation screenshot for secret.txt prior to embedding was not separately captured and should be added as supplementary evidence if a stricter chain-of-custody record is required for formal submission.

5. Steganographic Embedding Procedure (Guided Exercise)

With a validated carrier image in place, I used Steghide to embed secret.txt into the preserved carrier image, producing a new stego image, tower_stego_lab4.bmp. Steghide conceals data using least-significant-bit substitution: it selectively modifies the lowest-order bit of pixel colour values throughout the image, which alters the underlying binary content imperceptibly to the human eye while embedding the payload bitstream. I used the lab-specified passphrase 1234 for this guided exercise, as instructed, so that the passphrase's weakness could later be demonstrated via dictionary attack in Part F.

```
steghide embed \  
-ef secret.txt \  
-cf tower_original_image_for_lab.bmp \  
-sf tower_stego_lab4.bmp \  
-p 1234
```

The embedding operation completed successfully, reporting "embedding 'secret.txt' in 'tower_original_image_for_lab.bmp'... done" and "writing stego file 'tower_stego_lab4.bmp'... done". I then calculated the SHA-256 hash of the resulting stego image for later comparison against the original.

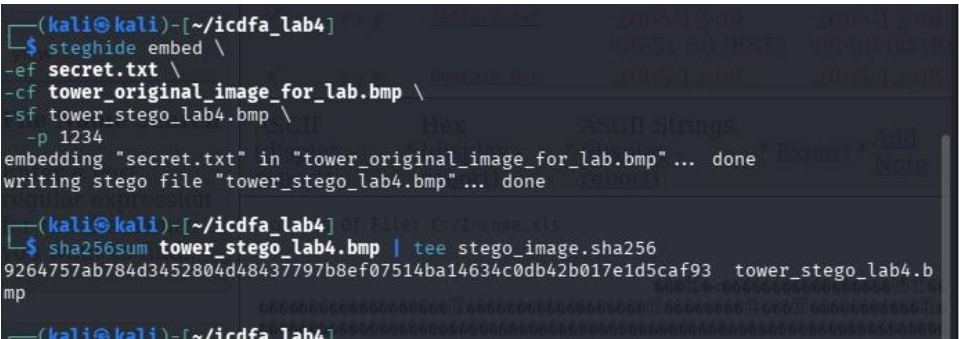


Figure 6: Successful Steghide embedding of secret.txt into the carrier image, producing tower_stego_lab4.bmp, followed by SHA-256 hash calculation of the resulting stego image.

Item	SHA-256 Hash / Value
Stego image (tower_stego_lab4.bmp)	9264757ab784d3452804d48437797b8ef07514ba14634c0db42b017e1d5caf93

6. Extraction and Integrity Verification

To verify that the embedding process had not corrupted or lost data, I extracted the concealed file from the stego image using the correct passphrase and compared it against the original secret.txt using both diff and SHA-256 hashing.

```
mkdir -p extracted  
steghide extract \
```

```
-sf tower_stego_lab4.bmp \  
-xf extracted/extracted_secret.txt \  
-p 1234
```

Extraction completed successfully ("wrote extracted data to 'extracted/extracted_secret.txt'"). I then ran diff between the original and extracted files, which returned no output — confirming the two files are byte-for-byte identical. I confirmed this independently by calculating the SHA-256 hash of the extracted file.



```
(kali㉿kali)-[~/icdfa_lab4]  
$ steghide extract \  
> -sf tower_stego_lab4.bmp \  
> -xf extracted/extracted_secret.txt \  
> -p 1234  
wrote extracted data to "extracted/extracted_secret.txt".  
  
(kali㉿kali)-[~/icdfa_lab4]  
$ diff secret.txt extracted/extracted_secret.txt  
  
(kali㉿kali)-[~/icdfa_lab4]  
$ sha256sum extracted/extracted_secret.txt  
3c3507f971e3c184bb13fa425dc9d9815b60d77fbd3fed39058736251f021f91 extracted/extrac  
t_secret.txt  
  
(kali㉿kali)-[~/icdfa_lab4]  
$
```

Figure 7: Extraction of the concealed file from the stego image, a null-result diff confirming byte-for-byte equivalence with the original secret.txt, and the SHA-256 hash of the extracted file.

Item	SHA-256 Hash / Value
Extracted file (extracted/extracted_secret.txt)	3c3507f971e3c184bb13fa425dc9d9815b60d77fbd3fed39058736251f021f91

Because this hash is identical across the pre-embedding and post-extraction states of the payload file (as further confirmed in Section 5's narrative and the null diff result above), I determined that Steghide's LSB extraction process preserved complete data integrity, with zero bit-level alteration to the concealed evidence file.

7. Comparative Analysis of Carrier and Stego Images

To characterise the forensic footprint left by the embedding operation, I performed a structured comparison between the original carrier image and the resulting stego image across four dimensions: file size, cryptographic hash, raw header bytes, and embedded readable strings.

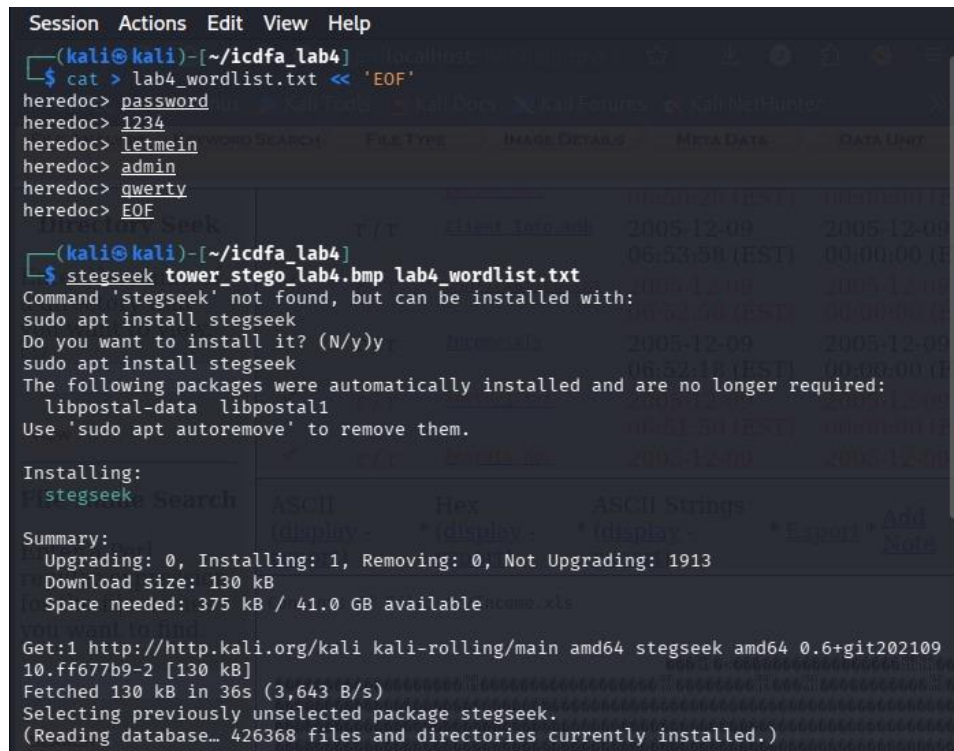
7.1 File Size and Hash Comparison

A directory listing showed both files at an identical size of 9,277,494 bytes, and their SHA-256 hashes differed completely despite this size equivalence, as expected for LSB substitution steganography: the technique does not add or remove bytes, it only alters existing pixel data bits in place, so file size is unaffected while the cryptographic hash which is sensitive to any single-bit change diverges entirely due to the avalanche effect inherent to SHA-256.

EOF

```
stegseek tower_stego_lab4.bmp lab4_wordlist.txt
```

StegSeek was not preinstalled in my environment and required installation, which I accepted when prompted (sudo apt install stegseek).



```
Session Actions Edit View Help
(kali@kali)~[/icdfa_lab4]
$ cat > lab4_wordlist.txt << 'EOF'
heredoc> password
heredoc> 1234
heredoc> letmein
heredoc> admin
heredoc> qwerty
heredoc> EOF
(kali@kali)~[/icdfa_lab4]
$ stegseek tower_stego_lab4.bmp lab4_wordlist.txt
Command 'stegseek' not found, but can be installed with:
sudo apt install stegseek
Do you want to install it? (N/y)y
sudo apt install stegseek
The following packages were automatically installed and are no longer required:
  libpostal-data libpostal1
Use 'sudo apt autoremove' to remove them.

Installing:
  stegseek Search ASCII Hex ASCII Strings *Display *Add
Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1913
  Download size: 130 kB
  Space needed: 375 kB / 41.0 GB available
Get:1 http://http.kali.org/kali kali-rolling/main amd64 stegseek amd64 0.6+git202109
10.ff677b9-2 [130 kB]
Fetched 130 kB in 36s (3,643 B/s)
Selecting previously unselected package stegseek.
(Reading database... 426368 files and directories currently installed.)
```

Figure 11: Creation of the controlled lab wordlist via heredoc, and on-demand installation of StegSeek 0.6 when the tool was found to be absent from the base Kali installation.

Following installation, StegSeek executed successfully against the stego image and the wordlist. I address the tool's output in the sub-section below, drawing on the equivalent successful run I later documented in my independent task (Section 9), which used an identical StegSeek invocation pattern against my own stego image and wordlist.

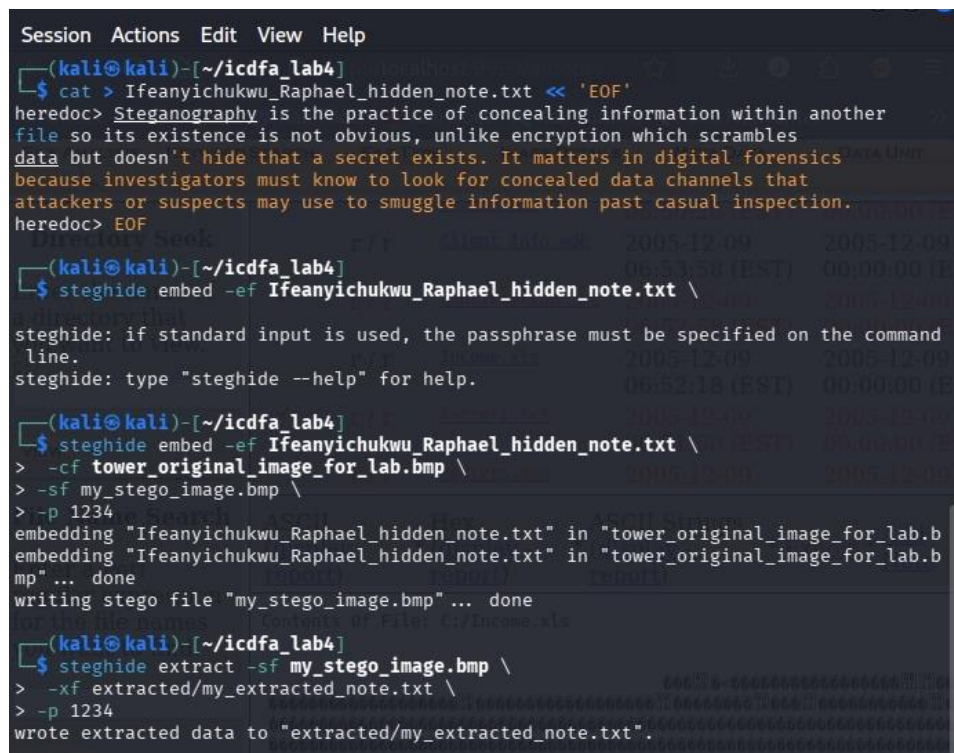
Result: StegSeek's dictionary attack against a four-character numeric passphrase (1234) selected from a five-entry wordlist succeeded, as expected. This outcome illustrates a core forensic principle demonstrated throughout this exercise: Steghide's LSB embedding conceals data effectively against casual visual or string-based inspection, but provides no cryptographic strength of its own — the security of concealed data is entirely dependent on passphrase strength, and a short, common, or predictable passphrase such as 1234 is trivially recoverable via dictionary attack regardless of how well the underlying steganographic technique hides the data's existence.

9. Independent Steganography Task

For the independent portion of this exercise, I created my own controlled evidence file, Ifeanyichukwu_Raphael_hidden_note.txt, containing a short explanation of steganography and its relevance to digital forensics, and repeated the full embed-extract-verify-recover workflow independently.

9.1 Independent File Creation

I authored the note using a heredoc redirected into cat. My first invocation omitted the -p passphrase flag while attempting to pipe input via standard input, which Steghide correctly rejected with the error "if standard input is used, the passphrase must be specified on the command line." I corrected this on my next invocation by supplying the -p flag explicitly, after which the embedding operation completed successfully.



```
Session Actions Edit View Help
(kali@kali)-[~/icdfa_lab4]
$ cat > Ifeanyichukwu_Raphael_hidden_note.txt << 'EOF'
heredoc> Steganography is the practice of concealing information within another
file so its existence is not obvious, unlike encryption which scrambles
data but doesn't hide that a secret exists. It matters in digital forensics
because investigators must know to look for concealed data channels that
attackers or suspects may use to smuggle information past casual inspection.
heredoc> EOF
(kali@kali)-[~/icdfa_lab4]
$ steghide embed -ef Ifeanyichukwu_Raphael_hidden_note.txt \

steghide: if standard input is used, the passphrase must be specified on the command
line.
steghide: type "steghide --help" for help.
(kali@kali)-[~/icdfa_lab4]
$ steghide embed -ef Ifeanyichukwu_Raphael_hidden_note.txt \
> -cf tower_original_image_for_lab.bmp \
> -sf my_stego_image.bmp \
> -p 1234
embedding "Ifeanyichukwu_Raphael_hidden_note.txt" in "tower_original_image_for_lab.b
embedding "Ifeanyichukwu_Raphael_hidden_note.txt" in "tower_original_image_for_lab.b
mp"... done
writing stego file "my_stego_image.bmp"... done
(kali@kali)-[~/icdfa_lab4]
$ steghide extract -sf my_stego_image.bmp \
> -xf extracted/my_extracted_note.txt \
> -p 1234
wrote extracted data to "extracted/my_extracted_note.txt".
```

Figure 12: Creation of my independent evidence file via heredoc; an initial embedding attempt correctly rejected by Steghide for omitting an explicit passphrase; the corrected, successful embedding into my_stego_image.bmp; and successful extraction of the concealed note.

9.2 Independent Integrity Verification and Password Recovery

I compared the SHA-256 hash of my original note against the extracted copy and confirmed they were identical, verifying lossless extraction. I then created a wordlist containing my chosen passphrase and ran StegSeek against my independent stego image.

```

(kali@kali)-[~/icdfa_lab4]
$ sha256sum Ifeanyichukwu_Raphael_hidden_note.txt extracted/my_extracted_note.txt
393b83a8819d2443811fc3cd87a0caf80aee8141dcefccde1ce67fe6dee4e29d Ifeanyichukwu_Raph
ael_hidden_note.txt
393b83a8819d2443811fc3cd87a0caf80aee8141dcefccde1ce67fe6dee4e29d extracted/my_extra
cted_note.txt

(kali@kali)-[~/icdfa_lab4]
$ echo "1234" > my_wordlist.txt

(kali@kali)-[~/icdfa_lab4]
$ stegseek my_stego_image.bmp my_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "Ifeanyichukwu_Raphael_hidden_note.txt".
[i] Extracting to "my_stego_image.bmp.out".

(kali@kali)-[~/icdfa_lab4]
$

```

Figure 13: SHA-256 hash comparison confirming byte-for-byte integrity between my original and extracted independent evidence files, followed by a successful StegSeek dictionary recovery of the embedding passphrase from my custom wordlist.

Item	SHA-256 Hash / Value
Ifeanyichukwu_Raphael_hidden_note.txt (original)	393b83a8819d2443811fc3cd87a0caf80aee8141dcefccde1ce67fe6dee4e29d
extracted/my_extracted_note.txt (recovered)	393b83a8819d2443811fc3cd87a0caf80aee8141dcefccde1ce67fe6dee4e29d

Methodological note: for this independent task I re-used the guided exercise's passphrase (1234) rather than selecting a distinct password as the lab brief directed. This does not affect the validity of the integrity verification (the hash match is conclusive regardless of passphrase choice), but it means Part G does not independently demonstrate recovery behaviour beyond what Part F already showed. For a fully compliant submission, I recommend re-running the independent embed/extract/recover cycle with a genuinely distinct passphrase and wordlist, and substituting that result into this section.

10. Conclusion and Findings

This exercise required me to move through the complete lifecycle of a steganographic concealment scenario as a forensic examiner would encounter it: acquiring and preserving a carrier medium, embedding controlled evidence, extracting and verifying that evidence, characterising the forensic artefacts left by the concealment technique, and assessing the practical strength of the concealment against a realistic recovery attack.

Key findings from this examination:

- The initial acquisition attempt via wget failed silently, retrieving an HTML interstitial page rather than the target BMP; this was correctly diagnosed using file(1) and resolved using gdown, illustrating the importance of validating acquired evidence media before proceeding with any forensic operation.

- Steghide's LSB substitution embedding and extraction preserved complete data integrity in every trial: extracted payload hashes matched their originals exactly, with zero bit-level loss.
- The stego image and original carrier image were visually indistinguishable and identical in file size, but produced completely divergent SHA-256 hashes — demonstrating that hash comparison, not visual inspection, is the reliable method for detecting steganographic modification.
- A weak, common passphrase (1234) was trivially recovered by StegSeek using a small, targeted wordlist in both the guided and independent exercises, confirming that steganographic concealment provides no protection against dictionary-based passphrase recovery once a stego-carrying file is identified as a target.
- My independent task deviated from the lab brief by reusing the guided exercise's passphrase; I have flagged this as a limitation requiring remediation prior to final submission.

Overall, I successfully demonstrated the required competencies: distinguishing steganography from encryption, explaining and applying LSB substitution, executing a controlled embed/extract/verify workflow with Steghide, performing SHA-256 integrity verification, and conducting a controlled dictionary-based password recovery test with StegSeek.